

Floating Point Number

What is a Floating Point Number?

A **floating point number** is a way to represent **real numbers** (including very small or very large values and decimals) using a format similar to scientific notation.

A

Scientific Notation

A number like 12345.6789 can be written as:

$$1.23456789 \times 10^4$$

In computers, this idea is used in binary form:

$$\pm \text{mantissa} \times \text{base}^{\text{exponent}}$$

- **Mantissa** (significand): the significant digits
 - **Base**: usually 2 (binary)
 - **Exponent**: moves the decimal (binary) point left or right
-

IEEE 754 Standard Overview

IEEE 754 is the **standard** for representing floating point numbers in binary on computers.

Common Formats

Format	Bits	Exponent Bits	Mantissa Bits	Approx. Decimal Precision
Single Precision	32	8	23	~7 digits
Double Precision	64	11	52	~15-16 digits

IEEE 754 Structure

32-bit (Single Precision)

S	Exponent (8 bits)	Fraction / Mantissa (23 bits)
1	8 bits	23 bits

- Sign bit (S): 0 for positive, 1 for negative
- Exponent: Stored with **bias** of 127
- Mantissa: Leading 1. is **implicit** (only the fractional part is stored)

64-bit (Double Precision)

S	Exponent (11 bits)	Fraction (52 bits)
1	11 bits	52 bits

- Bias for exponent = 1023
-

Normalization

Floating point numbers are usually stored in **normalized form**, meaning:

$$1.xxxx \times 2^n$$

Only the *xxxx* part (fraction) is stored, and the leading 1. is **assumed**.

Example: Decimal 6.25 $\rightarrow$ Binary = 110.01

Normalized form = 1.1001×2^2

Example: Storing 5.75 in IEEE 754 (32-bit)

1. Decimal to binary: $5.75 = 101.11 = 1.0111 \times 2^2$
2. Sign bit = 0 (positive)
3. Exponent = $2 + 127 = 129 \rightarrow 10000001$
4. Mantissa = 011100000000000000000000 (drop leading 1)

Result:

0 10000001 011100000000000000000000

Example: Python Float Representation

```
a = 0.1 + 0.2
print(a)  # Output: 0.30000000000000004
```

This is due to **floating point approximation** — 0.1 and 0.2 can't be exactly represented in binary.

Floating Point Approximation

Some decimal values **cannot be exactly represented** in binary.

Decimal	Binary Approximation	Exact?
0.1	0.000110011001100...	No
0.5	0.1	Yes
0.3	0.010011001100...	No

So:

```
print(0.1 + 0.2 == 0.3)  # False
```

To compare floats:

```
abs(a - b) < 1e-9  # Safer way to compare floats
```

Common Issues with Floating Points

- Rounding Errors
 - Loss of Precision
 - Comparison Failures
 - Accumulated Errors in Loops
-

Summary

- Floating point numbers use scientific notation (in binary).
 - IEEE 754 is the common standard (32-bit and 64-bit).
 - Precision issues arise due to binary approximation.
 - Always be cautious when comparing float values.
-

Visualization Diagram (ASCII)

IEEE 754 – 32-bit Example

S	Exponent (8 bits)	Fraction / Mantissa (23 bits)
0	10000001 (129)	01110000000000000000000 (for 5.75)

Normalization Visual

Decimal: 5.75
Binary: 101.11
Normalized: 1.0111×2^2
Stored: Mantissa = 0111..., Exponent = $2 + 127 = 129$

Introduction to Binary Floating Point Representation

Floating point representation is a way to store **real numbers** (i.e., numbers with fractional parts) in binary format. It is commonly used in computers to represent numbers that cannot be expressed as integers.

A floating-point number consists of three parts:

1. **Sign bit** (s): Determines whether the number is positive or negative.
2. **Exponent** (E): Specifies the magnitude of the number (i.e., how far to shift the binary point).
3. **Mantissa** (M): Contains the precision bits of the number.

General Formula for Floating Point Representation

$$N = (-1)^s \cdot 2^{E-\text{Bias}} \cdot (1.M)$$

Where:

- s : sign bit (1 for negative, 0 for positive)
 - E : exponent (stored with a bias)
 - Bias: depends on the precision (e.g., 127 for single precision, 1023 for double precision)
 - M : the mantissa or significand (with an implicit leading 1 in normalized form)
-

Conversion from Decimal to Binary Floating Point

Step-by-Step Process

1. **Sign bit** (s):
 - If the number is positive, $s = 0$
 - If the number is negative, $s = 1$
2. **Convert the integer part to binary:**
 - Divide the integer part by 2 repeatedly until the quotient is zero.
 - Collect the remainders in reverse order.
3. **Convert the fractional part to binary:**
 - Multiply the fractional part by 2 repeatedly.
 - Take the integer part of each result as the binary digit and continue multiplying the fractional part.

4. **Normalize the binary number:**

- Move the decimal point to create a normalized binary number (i.e., with a leading 1 before the point).
- Record how many times you moved the decimal point to determine the exponent.

5. **Calculate the exponent:**

- The exponent is the number of places the decimal point was moved, adjusted by the bias for the format (e.g., bias = 127 for 32-bit, 1023 for 64-bit).

6. **Mantissa:**

- The mantissa is the fractional part of the binary number after normalization, with the leading 1 omitted.

IEEE 754 Single Precision (32-bit) Format

Structure

- 1 bit for the sign: s
- 8 bits for the exponent: E (with bias 127)
- 23 bits for the mantissa: M

Formula:

$$N = (-1)^s \cdot 2^{E-127} \cdot (1.M)$$

Example: Convert -13.25 to IEEE 754 Single Precision

1. Sign bit: Since -13.25 is negative, the sign bit is 1.
2. Binary representation: Integer part 13 in binary is 1101, and fractional part .25 in binary is .01. Therefore, -13.25 in binary is -1101.01 .
3. Normalize: $-1101.01 \rightarrow -1.10101 \times 2^3$
4. Exponent: Bias for single precision = 127. Exponent = $3 + 127 = 130 \rightarrow$ Exponent in binary: 10000010
5. Mantissa: After the leading 1., the mantissa is 10101000000000000000000 (padded to 23 bits).

IEEE 754 Representation:

- Sign: 1
- Exponent: 10000010
- Mantissa: 10101000000000000000000

Final IEEE 754 32-bit representation:

1 10000010 101010000000000000000000

IEEE 754 Double Precision (64-bit) Format

Structure

- 1 bit for the sign: s
- 11 bits for the exponent: E (with bias 1023)
- 52 bits for the mantissa: M

Formula:

$$N = (-1)^s \cdot 2^{E-1023} \cdot (1.M)$$

Example: Convert -123.75 to IEEE 754 Double Precision

1. Sign bit: Since -123.75 is negative, the sign bit is 1.
2. Binary representation: Integer part 123 in binary is 1111011, and fractional part .75 in binary is .11. Therefore, -123.75 in binary is -1111011.11 .
3. Normalize: $-1111011.11 \rightarrow -1.11101111 \times 2^6$
4. Exponent: Bias for double precision = 1023. Exponent = $6 + 1023 = 1029 \rightarrow$ Exponent in binary: 10000000101
5. Mantissa: After the leading 1., the mantissa is 11101111000 (padded to 52 bits).

IEEE 754 Representation:

- Sign: 1
- Exponent: 10000000101
- Mantissa: 11101111000

Final IEEE 754 64-bit representation:

1 10000000101 1110111100

Conversion from IEEE 754 to Decimal

Given an IEEE 754 representation, the steps to convert back to decimal are:

1. Extract the sign, exponent, and mantissa from the binary string.
2. Calculate the exponent by subtracting the bias from the exponent field.
3. Reconstruct the mantissa by adding the leading 1 and then appending the mantissa bits.
4. Compute the result using the formula:

$$N = (-1)^s \times 2^{E-\text{Bias}} \times (1.M)$$

Practice Examples

Example 1: Convert 0.1 to IEEE 754 (32-bit)

1. Sign: Positive $\rightarrow s = 0$
2. Binary: 0.1 in binary is 0.0001100110011... (repeating)
3. Normalize: $0.1 = 1.1001100110011... \times 2^{-4}$
4. Exponent: Bias = 127 $\rightarrow$ Exponent = $127 - 4 = 123 \rightarrow$ Binary = 0111111011
5. Mantissa: 10011001100110011001100 (padded)

Final IEEE 754 32-bit representation:

0 0111111011 10011001100110011001100

Example 2: Convert 123456.789 to IEEE 754 (64-bit)

1. Sign: Positive $\rightarrow s = 0$
2. Binary: 123456.789 in binary $\approx 11110001000000000.110010100111111...$
3. Normalize: $1.1110001000000000110010100111111... \times 2^{16}$
4. Exponent: Bias = 1023 $\rightarrow$ Exponent = $1023 + 16 = 1039 \rightarrow$ Binary = 10000001111
5. Mantissa: 111000100000000011001010011111111110001110000101011 (padded)

Final IEEE 754 64-bit representation:

0 10000001111 111000100000000011001010011111111110001110000101011

Recap of Key Concepts

- **IEEE 754 Single Precision (32-bit):**
 - 1 sign bit, 8 exponent bits, 23 mantissa bits
- **IEEE 754 Double Precision (64-bit):**
 - 1 sign bit, 11 exponent bits, 52 mantissa bits
- **Formula:**

$$N = (-1)^s \cdot 2^{E-\text{Bias}} \cdot (1.M)$$
